



PRÉ-RENTRÉE 2026-2027

Séance 5 - Fiche enseignant

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Corrigé, différenciation et pilotage

Objectifs

- écrire une recherche séquentielle ;
- calculer extremum et moyenne ;
- comprendre un tri par insertion ;
- tracer une recherche dichotomique ;
- livrer un projet documenté et testé ;

Déroulé minute par minute

- 0-10 quiz ;
- 10-27 recherche/extremum ;
- 27-43 insertion ;
- 43-58 dichotomie/coût ;
- 58-63 brief projet ;
- 63-68 pause ;
- 68-105 projet ;
- 105-115 oral/tests ;
- 115-120 bilan ;

Rituel prêt à l'emploi

Question 1

Dans une liste non triée, quel algorithme simple permet de trouver la première occurrence d'une cible ? Que renvoie-t-on si elle est absente ?

Réponse attendue : Une recherche séquentielle ; on peut renvoyer `None` si la cible est absente.

Question 2

Quel prérequis est indispensable à la recherche dichotomique et que devient approximativement la zone de recherche à chaque étape ?

Réponse attendue : La liste doit être triée ; la zone restante est divisée approximativement par deux.

Notions de cours

Recherche séquentielle

Parcourt les éléments un à un. Coût proportionnel au nombre d'éléments.

Tri par insertion

Maintient une partie gauche déjà triée et insère l'élément courant à sa place. Le pire cas est quadratique.

Recherche dichotomique

Dans une liste triée, compare à l'élément central et élimine la moitié restante. Le nombre d'étapes croît comme un logarithme.

Correction et terminaison

- invariant : propriété vraie avant et après chaque tour ;
- variant : entier positif qui décroît vers zéro ;
- tests : cas présent, absent, premier, dernier, liste vide.

Activité commune - choisir l'algorithme

Besoin	Algorithme	Précondition	Coût intuitif
trouver un identifiant dans une liste quelconque			
trouver dans une liste triée très longue			
remettre une petite liste dans l'ordre			
trouver la plus grande température			

Projet final

À partir de `mesures_capteurs.csv`, produire un programme qui :

1. charge et valide les données ;
2. calcule moyenne, minimum et maximum ;
3. filtre les alertes ;
4. recherche un capteur par identifiant ;
5. trie les mesures ;
6. affiche un rapport lisible ;
7. contient des fonctions, docstrings et assertions.

Parcours Projet guidé - Ahmad

- utiliser le fichier `projet_final_starter_ahmad.py` ;
- compléter les fonctions dans l'ordre indiqué ;
- exécuter les tests fournis après chaque fonction ;
- expliquer deux fonctions lors de la restitution.

Parcours Projet autonome - Ahmed

- utiliser `projet_final_specification_ahmed.md` ;
- définir l'architecture du programme avant de coder ;
- écrire les tests, y compris liste vide et identifiant absent ;
- comparer recherche séquentielle et dichotomique ;
- expliquer la complexité intuitive et un invariant.

Corrigé essentiel

Le pack `09_PACKS_CODE/1re_nsi_CODE_ENSEIGNANT.zip` contient la solution de référence.
Une autre solution est recevable si elle respecte le contrat, les tests et la lisibilité.

Critères : exactitude, décomposition, tests, documentation, absence de duplication, gestion des cas limites et explication orale.

Consignes prêtes à dire

- « Avant d'exécuter, écris ce que tu prévois. »
- « Une réponse sans contrôle reste une hypothèse. »
- « Le bug utile est celui que l'on peut reproduire. »
- « Ne change qu'une chose à la fois, puis relance les tests. »
- « Explique le rôle de cette variable sans lire le code mot à mot. »

Points de vigilance

- ne pas transformer l'activité en copie de code projeté ;
- vérifier que les deux élèves alternent pilote et navigateur ;
- ne pas confondre réussite après aide D/E et autonomie ;
- demander un test sur les bornes ;
- conserver le fichier final et le journal des erreurs.

Indicateurs de réussite

Élève	Prévision exacte	Code exécutable	Tests pertinents	Explication	Aide maximale
Ahmad BELDI					
Ahmed BENHADJ SALEM					